

The Alberta Buck — Privacy

Paying Carol: private commerce, honest records, and nothing for Mallory

Perry Kundert

2026-09-24



Buy a coffee with a card and at least four businesses learn about it: the coffee shop, its payment processor, the card network and your bank. Each keeps the record, and several build products to sell from what they learn. Show your licence at a bar, a rental counter or a dispensary and a scanner may keep a copy. In September 2026 a criminal service offered searchable scans of more than 153 million American and Canadian driver's licences, about 1.1 million of them Canadian, taken from one such scanning company's systems over more than a year: photos, licence numbers, and a timestamp for every scan.¹

Meanwhile, those who launder money for a living buy their privacy with shell companies, nominees and couriers. The honest citizen is the one left standing in the light.

This paper shows another arrangement. Alberta certifies who you are once. After that you can prove you are a certified person without showing who, show a counterparty exactly who you are without showing everyone, and keep a receipt a court can rely on. Commerce between free citizens stays between them; the record exists for the people entitled to it.

We follow one payment. Bob owes Carol 100 BUCK. Mallory, who has no special access, watches everything public. We will build the payment one layer at a time, and every step below is running code: each fragment executes against a live local Ethereum chain, and its output is printed beneath it.² (PDF, Text)

¹B. Krebs, "FBI Probes Service Selling 153M+ Drivers Licenses," Krebs on Security, September 2026 (link); "ID verification giant IDScan confirms data breach," TechCrunch, 10 September 2026 (link).

²The credential construction awaits independent review, and the zero-knowledge proof setups used here are development setups that a deployment replaces with public ceremonies. The demonstration's world is built by `make nix-snark-privacy-fixtures` (`gen_privacy_world.py`); this paper renders with `make nix-venv-doc-privacy`. See Proofs and the receiving-key architecture.

The cast, and how to read this

- **Bob** and **Carol** are private citizens of Alberta. Bob pays; Carol is paid.
- **Mallory** sits in the middle of every picture. She sees whatever the blockchain shows anyone: every transaction, every account, every event. She holds nobody's keys.
- **Alberta's issuer** certifies identities. **Aspen Mutual Credit Union** is Bob's credit union and, like any bank, a public institution. Every name here is invented for the demonstration.

Each step has three parts. A short account of what happens. A code fragment showing the data gathered in that step, where each value comes from, and what it produces. And a picture: Bob on the left, Carol on the right, Mallory in the middle, with the service doing the work below. A *plus* adds material, a *lock* marks sealed contents, an *opening arrow* exposes contents to their intended reader, and a *check* marks a completed check. An arrow crossing Mallory's column gives her a view of the traffic, not a key.

Payloads are drawn as cards. Every field carries a badge saying who can read it:

Badge	Meaning
PUBLIC	anyone can read it, Mallory included
PRIVATE	known to the named parties only; never published
SEALED	encrypted: only the named reader can open it
SECRET	never leaves its holder, not even inside a proof
PROVEN	established by a proof without being shown

When the code prints a transaction as Mallory sees it, an account shows only as its address. The `[reader: ...]` annotation after it is for you, not for her. Public identities, such as the credit union, show their names because Mallory can look those up herself.

The demonstration starts a private chain with the real contracts: the identity registry, the BUCK token and the Notes pool. The zero-knowledge proofs that take a laptop 15 to 20 seconds each were generated in advance from exactly the inputs used below; everything else happens here.

```
import os, random
from alberta_buck.wallet import backend, cards
from alberta_buck.wallet.bn254 import G1, add, eq, mul, neg, rand_scalar
from alberta_buck.wallet.identity import canonical_identity_data, identity_scalar
from alberta_buck.wallet.issuer import Issuer
from alberta_buck.sim.anvil import Anvil
from alberta_buck.sim.cast import ASPEN, BOB, CAROL
from alberta_buck.sim.privacy_world import PrivacyChain, PrivacyWorld

assert backend() == "kernel", "must run the production kernel: evaluate in the repo venv"
_seed = random.Random(0x9A1B0B)          # one deterministic transcript
rng = lambda: _seed.getrandbits(256)

world = PrivacyWorld.load()               # the cast's wallets and the pre-made proofs
A, P = world.accounts, world.people      # accounts and people, by name
U = world.unit                           # BUCK carries 6 decimals
anvil = Anvil(chain_id=1, auto_impersonate=True, timestamp=1790000000).start()
alberta = Issuer.setup("alberta-identity", 0xA1BE27A00000000000000000000000000000000A1, rng=rng)
chain = PrivacyChain(anvil, world, alberta, rng=rng, block_time=12)

os.makedirs("images/privacy/cards", exist_ok=True)
card = lambda c, name: c.png(f"images/privacy/cards/{name}.png")
point = lambda fields: mul(G1, identity_scalar(canonical_identity_data(fields)))
print(f"chain {anvil.w3.eth.chain_id}: registry {chain.reg.address[:10]}..., "
      f"BUCK {chain.buck.address[:10]}..., Notes {chain.notes.address[:10]}...")

chain 1: registry 0x5FbDB231..., BUCK 0x5FC8d326..., Notes 0x9A9f2CCf...
```

First, a credential

I1. Carol proves who she is, once

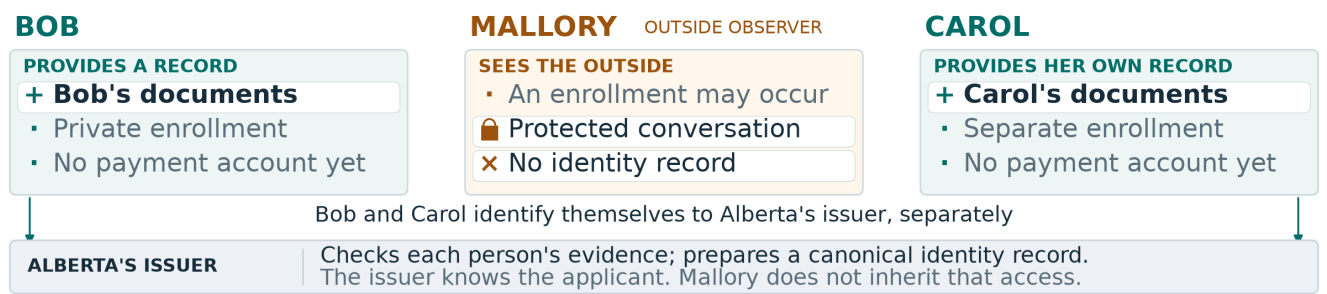
Carol visits Alberta’s issuer with her documents, as she would for a licence. The issuer checks them and writes a short *core record*: her name at first certification, date of birth, and a person number the registry never reuses. Bob does the same on another day.

The record is hashed into a single number, and from it a point on an elliptic curve: Carol’s *identity point*, **M**. It is a fingerprint. Given the record anyone can check it; given only **M** nobody can recover the record. Mallory might notice an appointment, but the conversation is private.

```
record = canonical_identity_data(CAROL)          # the core record, fixed at first certification
m = identity_scalar(record)                      # m = keccak(tag || record) mod the group order
print(record)
print(f"m = {hex(m)[:18]}.. and M = m*G: Carol's identity point, a fingerprint of the record")
card(cards.identity_record(CAROL, "Carol"), "record")

{"date_of_birth": "1994-10-27", "epoch": 42, "family_name": "Nakamura", "given_name": "Carol", "id_number": "AB-P-8812-5507", "id_type": "Albe
m = 0x78540a5aa5db6031.. and M = m*G: Carol's identity point, a fingerprint of the record
```

CORE IDENTITY RECORD			held by Carol
PRIVATE: Carol + issuer	given_name	Carol	certified at enrolment
PRIVATE: Carol + issuer	family_name	Nakamura	certified at enrolment
PRIVATE: Carol + issuer	jurisdiction	Alberta, Canada	certified at enrolment
PRIVATE: Carol + issuer	id_type	Alberta Identity	certified at enrolment
PRIVATE: Carol + issuer	id_number	AB-P-8812-5507	certified at enrolment
PRIVATE: Carol + issuer	date_of_birth	1994-10-27	certified at enrolment
PRIVATE: Carol + issuer	issuer_id	alberta-identity	certified at enrolment
PRIVATE: Carol + issuer	issued_at	2026-04-09T10:05:00Z	certified at enrolment
PRIVATE: Carol + issuer	epoch	42	certified at enrolment
Fixed at first certification: the identity point M is its hash, for life.			



I2. The issuer signs, and Carol keeps the signature

The issuer signs Carol’s number with its private key and hands her the record and the signature. Her wallet checks the signature against the issuer’s published key. The raw signature never leaves her wallet:

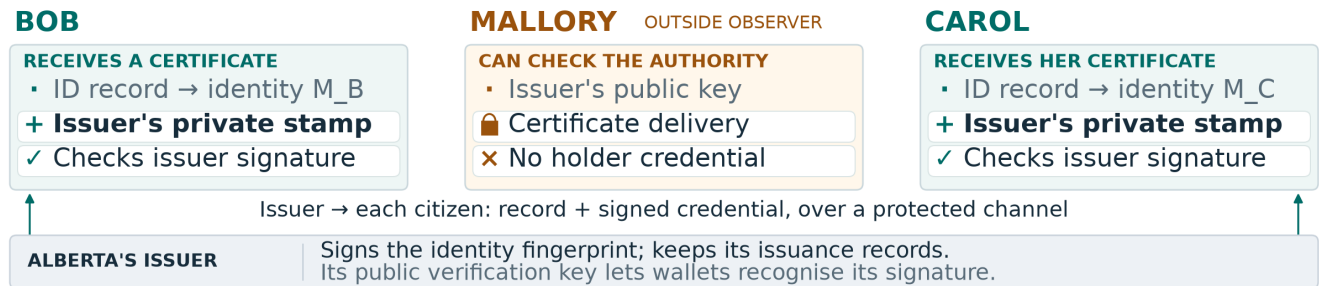
whenever she uses it, she shows a freshly masked version that proves the same thing and matches nothing she has shown before.

```
creds = {who: alberta.issue(fields, applicant_addr=0, rng=rng)          # no account exists yet
         for who, fields in (("bob", BOB), ("carol", CAROL), ("aspen", ASPEN))}
cred = creds["carol"]
print(f"the issuer signed Carol's m:          {cred.m == m}")
print(f"her wallet verifies the signature:    {alberta.verify_credential(cred)}")
card(cards.credential(cred, "Carol"), "credential")
```

```
the issuer signed Carol's m:          True
her wallet verifies the signature:    True
```

CREDENTIAL		issued to Carol	
PRIVATE: Carol + issuer	identity scalar m	0x7854..7377	hash of the core record
SECRET: Carol	signature	((0xb7f..f9a, 0xa5b..93c), ..)	Alberta's issuer's PS signature on m
PUBLIC	issuer key	published	trusted on chain by governance

The raw signature never leaves the wallet; only masked presentations do.



Details that change

Carol will move house, change her phone number, perhaps her name. None of that may change **M**, or every account and payment tied to it would lose its footing. So the core record holds only what never changes, and the current details go in a separate *particulars* certificate that the registry signs over **M**. Each detail is sealed on its own, so Carol can show a bar her photo and her age band, and nothing else. That is what an ID scan should have been.

When she moves, the registry issues version 2. **M** is untouched, and so is everything bound to it.³

```
from alberta_buck.registry.certificate import registry_keygen
from alberta_buck.registry.particulars import disclose, issue_particulars, verify_disclosure

registry = registry_keygen(rng)          # the registry's signing key
now = {"legal_name": "Carol Nakamura", "address": "1204 10 Ave SW, Calgary AB",
       "phone": "+1 403 555 0142", "photo": "sha256:5b1c0e9f", "age_band": "25-34"}
details = issue_particulars(registry, "alberta-identity", P["carol"].M, 1, "2026-04-09", now, rng)
at_the_door = disclose(details, "photo", "age_band")
print(f"the bar sees:", verify_disclosure(at_the_door, registry.pk, P["carol"].M))
moved = issue_particulars(registry, "alberta-identity", P["carol"].M, 2, "2026-09-01",
                          dict(now, address="77 Main St, Canmore AB"), rng)
```

³particulars.py; accumulator specification, section 8.4. Categorical facts such as an age band can also be proven as registry memberships, which reveal the category without any certificate.

```

print("after the move, the lender sees:", verify_disclosure(disclose(moved, "address"),
                                                            registry.pk, P["carol"].M))
print("and M is unchanged:", moved.certificate.M == details.certificate.M)
card(cards.particulars(details, "Carol", disclosed=("photo", "age_band")), "particulars")

```

```

the bar sees: {'photo': 'sha256:5b1c0e9f', 'age_band': '25-34'}
after the move, the lender sees: {'address': '77 Main St, Canmore AB'}
and M is unchanged: True

```

PARTICULARS v1			held by Carol
PRIVATE: Carol	address	0x6198..5a31	commitment only; value withheld
PUBLIC	age_band	25-34	disclosed; opens its commitment
PRIVATE: Carol	legal_name	0xf890..600c	commitment only; value withheld
PRIVATE: Carol	phone	0xd3d1..3e71	commitment only; value withheld
PUBLIC	photo	sha256:5b1c0e9f	disclosed; opens its commitment
Signed over M. A change is a new version; M and everything bound to it stay.			

13. Carol's wallet makes her keys

Carol's wallet starts from a secret seed only she holds. From it the wallet derives a *receiving key*, the mailbox that addressed payments will be sealed to. The receiving key is not derived from her identity: a photocopy of your identification should not open your mail.

The registry records that Carol's identity owns that mailbox, and two other facts about her, each as a sealed entry under its own random salt. It publishes only one number summarising them all, the *identity root*. The code rebuilds that root from nothing but Bob's and Carol's own wallet secrets.

```

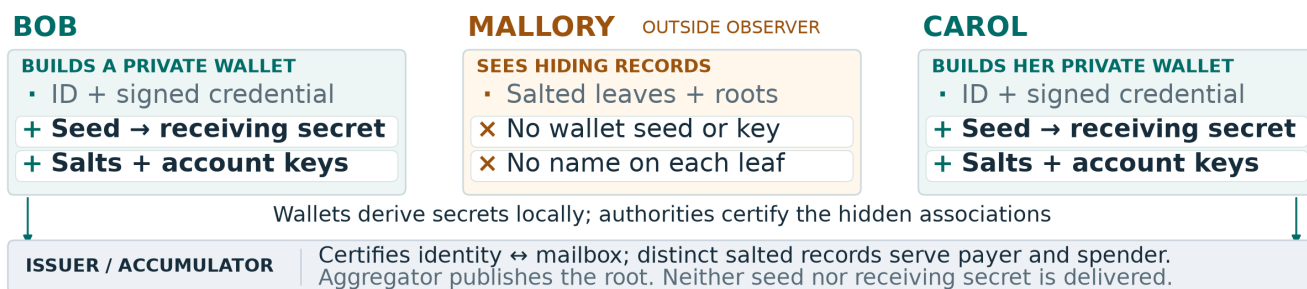
from alberta_buck.wallet.recvkey import receiving_key
from alberta_buck.wallet.salt import derive_salt
from alberta_buck.registry.merkle_service import rooted_registry
from alberta_buck.registry.tree import identity_leaf_salted, mailbox_leaf, receiving_leaf

bob, carol = P["bob"], P["carol"]
k, pk_recv = receiving_key(carol.seed)          # from her seed, never from her identity
salts = {purpose: derive_salt(carol.seed, world.kyc, n)
          for n, purpose in enumerate(("receiving", "mailbox", "naming"))}
print(f"receiving key and salts come from Carol's seed: {k == carol.k and salts == carol.salts}")

tree = rooted_registry("registry:kyc")          # the registry's private subtree
for leaf in (receiving_leaf(bob.m, bob.k, bob.salts["receiving"]),
             identity_leaf_salted(bob.M, bob.salts["naming"]),
             receiving_leaf(carol.m, k, salts["receiving"]),      # identity <-> receiving secret
             mailbox_leaf(carol.M, pk_recv, salts["mailbox"]),    # identity <-> public mailbox
             identity_leaf_salted(carol.M, salts["naming"])),     # identity, to be named by
            tree.insert_leaf(leaf)
print(f"identity root {hex(tree.root()[18])}.. (the proofs were made against it: "
      f"{tree.root() == world.identity_root})")

receiving key and salts come from Carol's seed: True
identity root 0x2243cb2b517b536a.. (the proofs were made against it: True)

```



I4. The wallet packages a fresh account

To use BUCK, Carol opens an account: an ordinary Ethereum address with its own signing key. Her wallet adds a second key pair, the account's *identity key*, and seals her identity point under it. Then it builds the registration package: the sealed identity, a freshly masked credential, and a proof that the sealed point is one Alberta certified and that Carol holds the key.

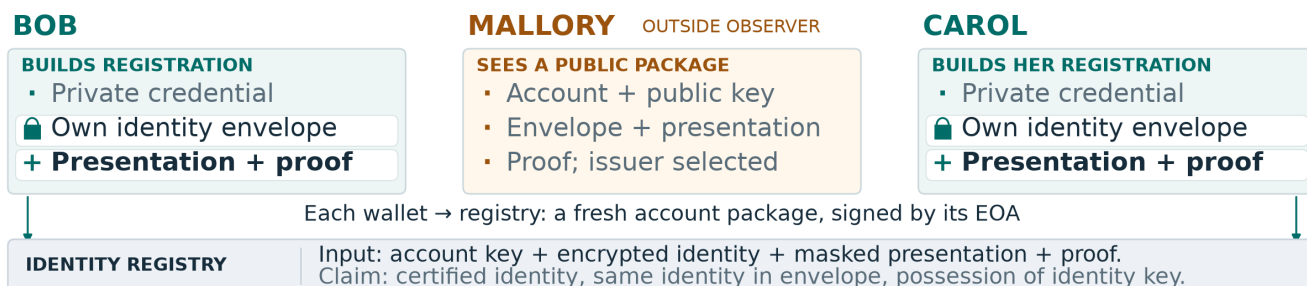
Nothing in the package says "Carol". It says "a certified person", and proves it.

```
pkg = chain.registration_package(A["carol"], cred, rng)
print(f"account {pkg.account.address}")
print(f"sealed identity E decrypts to Carol's M with the account key: "
      f"{eq(add(pkg.E.C, neg(mul(pkg.E.R, A['carol'].sk))), carol.M)})")
card(cards.registration(pkg, "Carol"), "registration")
```

```
account 0xf269DcD36ce4672f09f804e7b808F32029644F43
sealed identity E decrypts to Carol's M with the account key: True
```

REGISTRATION PACKAGE			built by Carol's wallet
PUBLIC	account	0xf269...4F43	a fresh Ethereum key
PUBLIC	identity key pk	(0x2f7...a97, 0xce0...ff5)	fresh key pair; sk stays
SEALED → Carol	identity envelope E	R 0x2cf...c5c C 0x1cd...017	Enc(M, pk): only sk opens it
PUBLIC	masked credential	A(0x305...194, 0x196...663)	signature re-masked: unlinkable
PROVEN	registration proof	e=0x1772...7698	certified M; E holds it; holds sk
SECRET: Carol	identity secret sk	never sent	proven, not shown

The proof shows the envelope holds a CERTIFIED identity -- not whose.



I5. The registry checks the proof, without opening anything

The registry contract verifies the proof: an Alberta credential, the identity sealed in this account, and the key held by whoever sent the transaction, all bound to this account on this chain. It stores the account's identity key and sealed identity. Carol opens a second account, for savings, from the same credential. Bob opens his.

Mallory sees three registrations. She cannot tell that two of them are the same person, and even if she later learned Carol's record she could not test these accounts against it.

```
reg = chain.register(pkg)
chain.register(chain.registration_package(A["carolSavings"], cred, rng))
chain.register(chain.registration_package(A["bob"], creds["bob"], rng))
print(chain.observe(reg).render())
chain.bind_public(A["aspen"])           # the credit union: a public identity
chain.post_identity_root(tree.root())    # the aggregator publishes the identity root
print(f"\nCarol's two accounts share nothing on chain:\n"
      f"  everyday {A['carol'].address} key {cards.short_point(A['carol'].pk)}\n"
      f"  savings  {A['carolSavings'].address} key {cards.short_point(A['carolSavings'].pk)}")
```

```
IdentityRegistry.register from 0xf269..4F43 [reader: Carol's everyday account]
tx 0x990c..6660 block 29 gas 463,430
issuer      0xa1Be..00a1 Alberta's issuer (public)
pk          curve point (0x2f746f..f13a97, 0xce0ad0..94fff5)
E           ciphertext
pres       masked credential
proof      registration proof
emits Registered
  account  0xf269..4F43 [reader: Carol's everyday account]
  issuer   0xa1Be..00a1 Alberta's issuer (public)
```

```
Carol's two accounts share nothing on chain:
everyday 0xf269DcD36ce4672f09f804e7b808F32029644F43 key (0x2f7..a97, 0xce0..ff5)
savings  0x11c38344EF77494C5e5eAa11eC72fE9dD5e0947F key (0x296..0e9, 0x1f0..40c)
```

BOB

KEEPS; CAN REPEAT LOCALLY

- ID + credential + seed

✓ Registered account B

+ Fresh account B-next

MALLORY OUTSIDE OBSERVER

CAN VERIFY, NOT OPEN

- B and C are registered
- B-next is registered
- ✗ No identity equality test

CAROL

KEEPS; CAN REPEAT LOCALLY

- ID + credential + seed

✓ Registered account C

+ Fresh account C-next

Registry validates the package; only the checked account record remains in state

IDENTITY REGISTRY

Checks issuer, proof, key ownership and the exact registration context.
Stores key + ciphertext + issuer. Public transaction history keeps the proof.

Paying Carol directly

The simplest payment is a transfer from Bob's account to Carol's. The chain will show it: two addresses, an amount, a time. What it will not show is who owns them. BUCK adds one rule: before money moves between two private accounts, each party must have given the other a sealed copy of its identity. Nobody gets paid by a stranger they cannot name.

First, Bob's money. Under BUCK a person's credit lives in an account of its own, and how that works is another paper. Bob moves 500 BUCK of spending money from his credit account to his everyday account, the one he pays from. That transfer is public too: anyone who can tie his credit account to him can tie this one. He could have moved it privately with the A2 cheque at the end of this paper.

```

credit = world.new_account("bob", "bobCredit", rng)          # a second account, same Bob
chain.observer.label(credit.address, "Bob's credit account")
chain.register(chain.registration_package(credit, creds["bob"], rng))
chain.fund(credit, 1_000 * U)                                # credit: outside this paper
chain.fund(A["aspen"], 5_000 * U)                            # the credit union's own funds
for a, b in ((credit, A["bob"]), (A["bob"], credit)):
    chain.approve(chain.envelope(a, b.address, b.pk, rng)) # sealed identities: next step
chain.transfer(credit, A["bob"].address, 500 * U)
print(f"Bob's everyday account holds {chain.balance(A['bob'].address) / U:.2f} BUCK")

```

Bob's everyday account holds 500.00 BUCK

E1. Bob seals his identity for Carol

Bob's wallet reads Carol's public identity key from the registry and seals his own identity point under it. It adds a proof that the sealed point is the same one his account registered, and asks BUCK to record it with an allowance of zero: this says who Bob is, and gives Carol no right to spend his money.

```

to_carol = chain.envelope(A["bob"], A["carol"].address, A["carol"].pk, rng)
card(cards.envelope(to_carol, "Bob", "Carol"), "envelope")

```

IDENTITY ENVELOPE (approve)			Bob → Carol
PUBLIC	counterparty	0xf269..4F43	the account being approved
PUBLIC	allowance	0.00 BUCK	0: identity only, no spending
SEALED → Carol	envelope	R 0x218..56a C 0x1e5..684	Enc(M _B , pk _C)
PROVEN	re-encryption proof	e=0x2fae..66ed	same M as the registered envelope
Proves: same identity as the sender's registration, sealed for the reader.			

BOB

- ADDS AN IDENTITY ENVELOPE
 - Registered account B
 - M_B for Carol's key
 - + Same-identity proof

MALLORY OUTSIDE OBSERVER

- SEES THE REQUEST
 - B → C approval
 - Ciphertext + proof
 - ✗ Cannot read M_B

CAROL

- PROVIDES A PUBLIC LOCK
 - Registered account C
 - Public identity key
 - Private key stays here

Bob → BUCK: identity approval for Carol, with spending allowance zero

BUCK / REGISTRY

Reads both registered keys and Bob's registered ciphertext.
Bob's new envelope must contain that same identity and use Carol's key.

E2. The contract checks it and keeps a fingerprint

The contract checks the proof against Bob's existing registration, not against anything the transaction could substitute, and records a fingerprint of the sealed envelope. Mallory can copy the envelope from the transaction. She cannot open it, and she cannot swap in someone else's identity and keep the proof valid.

```
print(chain.observe(chain.approve(to_carol, 0)).render())
```

```

Buck.approve from 0x01f9..564d [reader: Bob's everyday account]
tx 0x6d60..76f3 block 49 gas 176,547
spender 0xf269..4F43 [reader: Carol's everyday account]

```

```

amount          0.000000 BUCK
E_bob           ciphertext
pi_CP           re-encryption proof
emits ApproveReceipt
  owner         0x01f9..564d [reader: Bob's everyday account]
  spender       0xf269..4F43 [reader: Carol's everyday account]
  receiptHash   0xfd16..3973
emits CarryingFrozen
  target        0xf269..4F43 [reader: Carol's everyday account]
emits Approval
  owner         0x01f9..564d [reader: Bob's everyday account]
  spender       0xf269..4F43 [reader: Carol's everyday account]
  value         0.000000 BUCK

```

BOB

KEEPS THE ORIGINAL

- Own identity envelope
- Envelope for Carol
- ✓ Approval accepted

MALLORY OUTSIDE OBSERVER

CAN COPY THE EVIDENCE

- Approval transaction
- Envelope fingerprint
- ✗ Cannot substitute a name

CAROL

RECEIVES PROVEN CIPHERTEXT

- 🔒 M_B for Carol's key
- ✓ Same-identity proof
- + Recorded fingerprint

Contract checks the proof; Carol retrieves the envelope from public calldata

BUCK / REGISTRY

Checks sender-key possession + equal identity + fresh encryption relation.
Stores the envelope's fingerprint as B → C receipt evidence.

E3. Carol opens it and checks the name

Carol's account key opens the envelope, and out comes Bob's identity point. Bob sends his core record over a private channel. Carol hashes the record and compares: the proof says *this is the identity behind Bob's account*, and the record says *whose identity it is*. Both must agree. A record for anyone else, even one letter different, does not match.

```

from alberta_buck.wallet.elgamal import elgamal_decrypt
opened = elgamal_decrypt(to_carol.E, A["carol"].sk)
print(f"sealed point is Bob's record:      {eq(opened, point(BOB))}")
print(f"...and not 'Robert Tremblay':      {eq(opened, point(dict(BOB, given_name='Robert')))}")

```

```

sealed point is Bob's record:      True
...and not 'Robert Tremblay':      False

```

BOB

DELIVERS THE WORDS

- Approved envelope
- 🔒 Bob's identity record
- Credential stays here

MALLORY OUTSIDE OBSERVER

MAY CARRY THE MESSAGE

- Traffic and endpoints
- 🔒 Encrypted record
- ✗ No name or ID number

CAROL

OPENS; THEN COMPARES

- ↳ Envelope → M_B
- ↳ Record → name + ID
- ✓ Record matches M_B

Bob → Carol: protected identity data; Carol joins the data to the public proof →

CAROL'S WALLET

Decrypt the identity point; fingerprint the received record; compare.
Matching data supplies the name. The point alone is not a name lookup.

E4. Carol returns the favour

Carol seals her identity for Bob the same way, and sends him her record. Now each has checked the other. No money has moved: recognising someone is not paying them.

```

to_bob = chain.envelope(A["carol"], A["bob"].address, A["bob"].pk, rng)
chain.approve(to_bob, 0)
print(f"Bob opens Carol's envelope to her record: "
      f"{eq(elgamal_decrypt(to_bob.E, A['bob'].sk), point(CAROL))}")

```

Bob opens Carol's envelope to her record: True

BOB

OPENS CAROL'S RESPONSE

- B → C evidence

↳ M_C + Carol's record

✓ Both directions match

MALLORY OUTSIDE OBSERVER

SEES THE REVERSE APPROVAL

- C → B approval

🔒 Carol's identity

✗ Cannot use Bob's key

CAROL

ADDS HER RETURN ENVELOPE

- Verified Bob's record

🔒 M_C for Bob's key

+ **Proof + data delivery**

← Carol repeats the checked approval with allowance zero; Bob checks her data →

BUCK / BOTH WALLETS

Contract records C → B evidence after verification; Bob opens and compares. Both identity directions are now established. No BUCK has moved.

E5. Bob pays; both keep a receipt

Bob's account sends 100 BUCK to Carol's. The contract finds both envelopes in place and moves the money. Mallory sees two addresses, 100 BUCK and a time; if she ever learns who owns either address, she learns this payment too. What she never gets is either name from the chain.

Carol's wallet prints a receipt naming both parties. Its proofs let anyone holding the receipt check it without trusting Carol, which is what makes it evidence rather than a claim.

```

from alberta_buck.wallet.render import TextDriver, render_receipt
pay = chain.transfer(A["bob"], A["carol"].address, 100 * U)
print(chain.observe(pay).render())
print()
print(TextDriver(48).render(render_receipt(chain.eoa_receipt(to_carol, A["carol"], pay, 100 * U))))

```

```

Buck.transfer from 0x01f9..564d [reader: Bob's everyday account]
tx 0x7503..f0a6 block 51 gas 88,363
to 0xf269..4F43 [reader: Carol's everyday account]
amount 100.000000 BUCK
emits Transfer
from 0x01f9..564d [reader: Bob's everyday account]
to 0xf269..4F43 [reader: Carol's everyday account]
value 100.000000 BUCK
emits BuckTransferReceipt
from 0x01f9..564d [reader: Bob's everyday account]
to 0xf269..4F43 [reader: Carol's everyday account]
amount 100.000000 BUCK
fromCipherHash 0xd056..cafb
toCipherHash 0xfd16..3973

```

ALBERTA BUCK PAYMENT RECEIPT

Receipt trh7zgwwavvu

FROM (payer) (PRIVATE IDENTITY)

Bob Tremblay

Alberta Identity - Alberta, Canada

acct 0x01f9fec6bb2f227da0fe180651c1616e0f03564d

idpt 0x110f449d8e6d..6a6925e3ced6

TO (payee - you) (PRIVATE IDENTITY)

Carol Nakamura

Alberta Identity - Alberta, Canada
 acct 0xf269dcd36ce4672f09f804e7b808f32029644f43
 idpt 0x1eb063312673..8a97e1de516b

TRANSACTION

Type EOA xfer (Identity-bound, private payer)
 Amount 100.000000 BUCK
 When 2026-09-21 14:17 UTC
 Chain 1
 Block 51
 Log 0
 Tx 0x75037ada8da8..b09997daf0a6

VERIFICATION

status @ issue: VALID
 receipt trh7zgwwavvu

a record, not a bearer instrument
 github.com/pjkundert/alberta-buck

BOB

AUTHORISES THE PAYMENT

- Both identity records
- + Signed transfer: 100
- ✓ Payment record saved

MALLORY OUTSIDE OBSERVER

READS THE PUBLIC PAYMENT

- B → C: 100 BUCK
- Time + account activity
- 🔒 Private identity contents

CAROL

RECEIVES; SAVES EVIDENCE

- Verified Bob's record
- ✓ 100 BUCK credited
- + Receipt + chain anchor

Bob → BUCK: transfer 100 to C; identities authorise disclosure, EOA signs the spend

BUCK CONTRACT

Checks both registered accounts, both receipt directions and available funds.
 Executes transfer; emits payment and receipt references. Names remain off chain.

Notes: breaking the visible link

A direct transfer publishes an edge from Bob's account to Carol's. *Notes* remove it. Money goes into a shared pool as a sealed note, and comes out later to whoever can prove the right to it. The pool sees many notes go in and many come out, and nothing on the chain says which is which.

Notes come in three kinds, and we will pay Carol with each: a *bearer* note from a public issuer (B1), a note *addressed* to Carol from a public issuer (A1), and a note addressed to Carol from a *private* issuer (A2). They are alternatives, not stages.

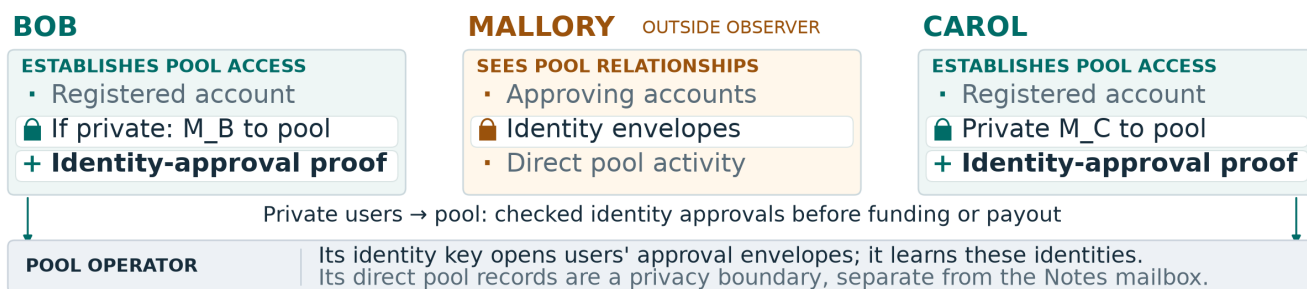
N1. Introductions to the pool

The pool is a contract with a public identity and an operator. Money into and out of it follows the ordinary rule, so private accounts give the pool their sealed identities first, just as Bob and Carol did for each other. The operator can open those envelopes: it knows who uses the pool. It cannot see which note is whose.⁴

```
for acct in (A["carol"], A["carolSavings"]):
    to_pool = chain.envelope(acct, chain.notes.address, chain.pool_pk, rng)
    chain.approve(to_pool, 0)
print(f"the operator's key opens Carol's envelope: "
      f"{eq(egamal_decrypt(to_pool.E, chain.pool_sk), carol.M)}")
```

the operator's key opens Carol's envelope: True

⁴Notes: the pool account. The operator's ability to open pool approvals is separate from Carol's receiving secret.



B1: cash from the credit union

A B1 note is digital cash with a known maker, like a banknote or a cashier's cheque. Its issuer is a public institution, here Aspen Mutual, and anyone holding the note's secret can cash it. Bob never appears on the chain at all.

B1.1. Bob buys a note

Bob pays Aspen Mutual 100 BUCK from his account, sealing his identity for the credit union first. It knows him anyway: he is a member. Aspen Mutual adds his note to a batch of four it is issuing for different members. The note is a value and a random secret, locked into a *commitment*: a fingerprint that can be published without revealing either.

```
chain.approve(chain.envelope(A["bob"], A["aspen"].address, A["aspen"].pk, rng))
print(chain.observe(chain.transfer(A["bob"], A["aspen"].address, 100 * U)).render())
b1 = world.notes["b1"]
print("\nAspen Mutual's batch; only Aspen Mutual and each holder know the values:")
for i, (o, cm) in enumerate(zip(b1.batch, b1.cms)):
    mine = " <- Bob's" if i == b1.raw["index"] else ""
    print(f" note {i} {o.v / U:>7.2f} BUCK commitment {hex(cm)[:14]}..{mine}")
card(cards.bearer_note(b1.opening, b1.cm, "Aspen Mutual Credit Union", "Bob"), "bearer-note")
```

```
Buck.transfer from 0x01f9..564d [reader: Bob's everyday account]
tx 0x5573..3074 block 55 gas 107,749
to 0x74c4..CBD7 Aspen Mutual Credit Union (public)
amount 100.000000 BUCK
emits Transfer
from 0x01f9..564d [reader: Bob's everyday account]
to 0x74c4..CBD7 Aspen Mutual Credit Union (public)
value 100.000000 BUCK
emits BuckTransferReceipt
from 0x01f9..564d [reader: Bob's everyday account]
to 0x74c4..CBD7 Aspen Mutual Credit Union (public)
amount 100.000000 BUCK
fromCipherHash 0xa3b8..7eef
toCipherHash 0x4bdd..672e
```

```
Aspen Mutual's batch; only Aspen Mutual and each holder know the values:
note 0 20.00 BUCK commitment 0x2f490a892e5a..
note 1 250.00 BUCK commitment 0x18d95ff603b5..
note 2 100.00 BUCK commitment 0x12f17f131e50.. <- Bob's
note 3 60.00 BUCK commitment 0x1d362b2fc8b4..
```

BEARER NOTE (B1)		issued by Aspen Mutual Credit Union, held by Bob	
PRIVATE: Bob	face value	100.00 BUCK	shown only when cashed
PUBLIC	issuer	Aspen Mutual Credit Union	authenticated by its batch signature
SECRET: Bob	bearer secret	ESGA-NODE-2YUM-IE0E-Q3P5-JBSZ...	the opening; never published
PUBLIC	commitment	0x12f1..0ef7	in the note tree since the mint
SECRET: Bob	spent marker	not yet shown	derived from the secret

Whoever holds the bearer secret can cash it: treat it like cash.

BOB

BUYS A BEARER NOTE

+ Pays 100 from everyday

His M sealed for Aspen

- A member: already known

MALLORY

OUTSIDE OBSERVER

SEES A PAYMENT TO A BANK

- Account → Aspen Mutual
- 100 BUCK + time
- ✗ No note, no secret

CAROL

NOT YET INVOLVED

- Own registered accounts
- Own private identity
- No mailbox supplied

Bob pays Aspen Mutual 100 BUCK; it adds his note to a batch of four

ASPEN MUTUAL

A public institution that knows its member, as any bank does.
Bob's note: 100 + a random secret, locked in a commitment.

B1.2. The credit union mints the batch

Aspen Mutual submits the four commitments, its signature over the batch, and a proof that the commitments are well formed and add up to what it is paying in. The contract checks all three and moves the money into the pool in the same step: either the notes exist and are funded, or nothing happened.

Mallory sees Aspen Mutual issue four notes worth 430 BUCK in total. She does not see the four values, who asked for them, or any secret.

```
chain.allow(A["aspen"], chain.notes.address, 10**15)
b1_mint = chain.mint_note(b1)
print(chain.observe(b1_mint).render())
print(f"\nthe batch proof was made in advance in {b1.timings['mint_prove_s']:.1f}s")
```

```
Notes.mint from 0x74c4..CBD7 Aspen Mutual Credit Union (public)
tx 0x07d0..11f1 block 57 gas 645,959
proof 256 bytes (Groth16 proof)
oldRoot 0xe0d753..b6cd7f
newRoot 0x1c024f..66f7bb
nextLeafIndex_ 0
totalFace 430.000000 BUCK
cms [0x2f490a..5d4330, 0x18d95f..2013f1, 0x12f17f..330ef7, 0x1d362b..f90710]
issuerMode [1, 1, 1, 1]
issuerSig issuer signature
emits Transfer
  from 0x74c4..CBD7 Aspen Mutual Credit Union (public)
  to 0x9A9f..63AE Notes pool (public)
  value 430.000000 BUCK
emits BuckTransferReceipt
  from 0x74c4..CBD7 Aspen Mutual Credit Union (public)
  to 0x9A9f..63AE Notes pool (public)
  amount 430.000000 BUCK
  fromCipherHash 0x9e6f..4aaf
  toCipherHash 0x9de4..f456
emits Minted
  issuer 0x74c4..CBD7 Aspen Mutual Credit Union (public)
```

```

newRoot      0x1c024f..66f7bb
totalFace    430.000000 BUCK
startIndex    0
count        4
emits IssuerBound
  issuer      0x74c4..CBD7 Aspen Mutual Credit Union (public)
  newRoot     0x1c024f..66f7bb

```

the batch proof was made in advance in 2.0s

BOB

HOLDS A BEARER NOTE

- + **100 + secret, from Aspen**
- Nothing of his on chain
- ✓ Note funded in the pool

MALLORY OUTSIDE OBSERVER

READS THE MINT

- Aspen: 4 notes, 430
- Four commitments
- ✗ No values, no buyers

CAROL

CAN VERIFY IT LATER

- Own wallet
- + **Public mint record**
- No payment yet

Aspen Mutual → Notes: a funded batch of four notes, Bob's among them

NOTES / BUCK

Checks the batch proof, Aspen Mutual's signature and public standing.
Moves 430 into the pool and adds the commitments in one step.

B1.3. Bob hands Carol the note

Bob gives Carol the note: printed as a code, sent in a message, it does not matter. Her wallet rebuilds the commitment from the secret, finds it in Aspen Mutual's batch, confirms the chain records Aspen Mutual as its issuer, and checks it has not been cashed.

This is cash, with cash's weakness: anyone who copies the secret can race Carol to cash it. So B1 suits handing over in person and cashing promptly. The addressed notes below close that gap.

```

from alberta_buck.wallet.notes import note_commitment, nullifier
note = b1.opening                                # what Bob handed over
cm = note_commitment(note)
print(f"bearer code {cards.bearer_code(note.rho)}")
print(f"rebuilds a commitment Aspen Mutual minted: {cm in b1.cms}")
print(f"the chain names its issuer as Aspen Mutual: "
      f"{chain.notes.functions.publicIssuerOfCommitment(cm).call() == A['aspen'].address}")
print(f"not yet cashed: "
      f"{not chain.notes.functions.nullifiers(nullifier(note.rho, note.id_hash)).call()}")

```

```

bearer code  ESGA-NODE-2YUM-IEOE-Q3P5-JBSZ-KQ6V-JUBX-P2FA-YL4X-TYU7-TZX6-QWQQ
rebuilds a commitment Aspen Mutual minted:  True
the chain names its issuer as Aspen Mutual:  True
not yet cashed:                               True

```

BOB

HANDS OVER THE NOTE

- Funded commitment
- + **Bearer code + mint ref**
- Could keep a copy

MALLORY OUTSIDE OBSERVER

INTERCEPTION MATTERS

- ✗ Copied code = cash
- Could race to cash it
- Hand it over in person

CAROL

CHECKS THE NOTE

- ↳ 100 + bearer secret
- ✓ Commitment in batch
- ✓ Not yet cashed

Bob → Carol: the note's secret, in person or by protected message →

CAROL'S WALLET

Rebuilds the commitment; checks issuer, funding and spent status.
A copy is spendable too: B1 is cash, to be cashed promptly.

B1.4. Carol seals her identity for the issuer

To cash a note, Carol's wallet derives its *spent marker* from the secret, which lets the chain refuse a second cashing without learning which note it was. It proves she holds a note in the tree. And, because B1 notes

come from a public institution, it seals her identity for Aspen Mutual and proves the sealed identity is the certified one behind her account.

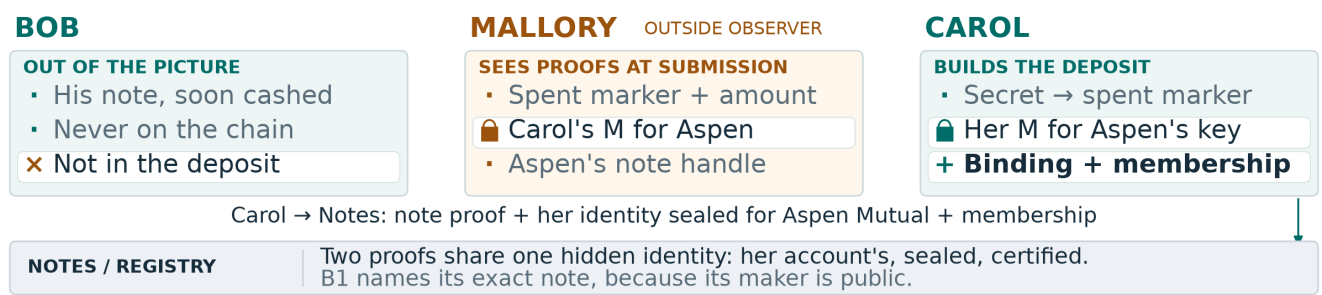
That last part is the rule of law in miniature: the note carried no name, but whoever cashes it is named to the institution that issued it, and to nobody else.

```
from alberta_buck.wallet.b1_binding import b1_bind_prove
draws = world.replay("b1", "depositorBinding") # the draws the pre-made proofs were built on
blind = rand_scalar(draws)
binding, sealed_for_aspen = b1_bind_prove(carol.m, A["carol"].sk, A["carol"].E, A["aspen"].pk,
                                         account=A["carol"].addr, chainid=1, b=blind, rng=draws)

F = cards.Field
card(cards.Card("B1 DEPOSIT: WHAT CAROL PROVES", "proved by Carol's wallet", note="Two proofs "
    "share one hidden identity, so neither can be about someone else.").add(
    F("spent marker", cards.short_int(b1.nullifier), "public", "", "refuses a second cashing"),
    F("value, payout", f"100.00 BUCK -> {cards.short_addr(A['carol'].address)}", "public"),
    F("note", cards.short_int(b1.cm), "public", "", "B1 shows its commitment: public maker"),
    F("identity, sealed", cards.short_ct(sealed_for_aspen), "sealed", "Aspen Mutual",
    "Carol's M for the issuer"),
    F("note secret, path", "used, not shown", "proven", "", "she holds a funded note"),
    F("certified identity", "used, not shown", "proven", "", "sealed M is her account's"),
    F("account key", "used, not shown", "secret", "Carol", "never leaves her wallet")),
    "b1-deposit")
```

B1 DEPOSIT: WHAT CAROL PROVES			proved by Carol's wallet
PUBLIC	spent marker	0x288b..6480	refuses a second cashing
PUBLIC	value, payout	100.00 BUCK -> 0xf269..4F43	
PUBLIC	note	0x12f1..0ef7	B1 shows its commitment: public maker
SEALED → Aspen Mutual	identity, sealed	R 0x1d7..a86 C 0x40c..cb5	Carol's M for the issuer
PROVEN	note secret, path	used, not shown	she holds a funded note
PROVEN	certified identity	used, not shown	sealed M is her account's
SECRET: Carol	account key	used, not shown	never leaves her wallet

Two proofs share one hidden identity, so neither can be about someone else.



B1.5. The pool checks everything and pays

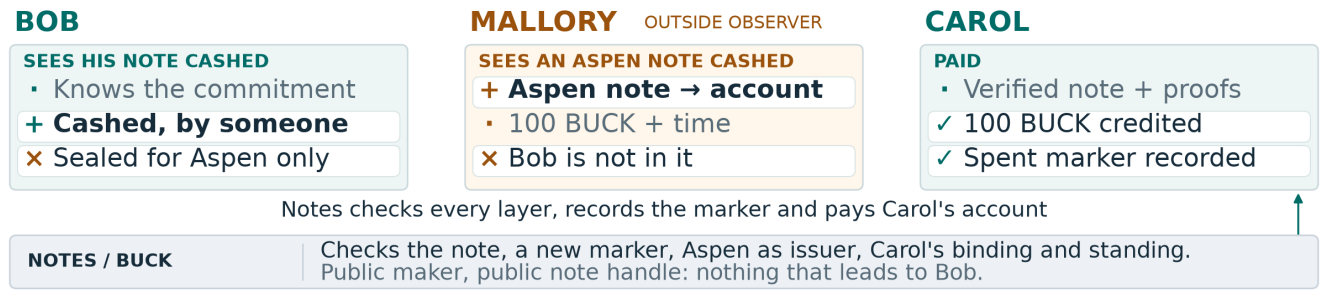
The contract checks the note proof, that the spent marker is new, that Aspen Mutual really issued this note, the identity binding, and that Carol's identity is currently certified. Then it records the marker and pays her 100 BUCK. A second attempt fails.

A B1 cashing names the exact note it redeems, because a public maker's notes are meant to be traceable to their maker. Mallory can see that one of Aspen Mutual's notes was cashed into this account. Bob is

nowhere in it.⁵ Only coincidence connects him: an earlier 100 BUCK payment from his account to the credit union, and a 100 BUCK note cashed later. Among a credit union's thousands of members that says little; in a demonstration with one, it says a lot. Standard amounts blur it further, as the last section explains.

```
b1_spend = chain.spend_note(b1)
print(chain.observe(b1_spend).render())
```

```
Notes.spendCoupledB1 from 0xf269..4F43 [reader: Carol's everyday account]
tx 0x21d4..a17d block 58 gas 763,103
proof 256 bytes (Groth16 proof)
root 0x1c024f..66f7bb
identityRoot 0x2243cb..b184cb
nullifier 0x288b62..956480
face 100.000000 BUCK
recipient 0xf269..4F43 [reader: Carol's everyday account]
issuanceCommitment 0x12f17f..330ef7
issuer 0x74c4..CBD7 Aspen Mutual Credit Union (public)
eDepForIss ciphertext
b1Proof depositor binding proof
membershipProof 256 bytes (Groth16 proof)
emits Transfer
  from 0x9A9f..63AE Notes pool (public)
  to 0xf269..4F43 [reader: Carol's everyday account]
  value 100.000000 BUCK
emits BuckTransferReceipt
  from 0x9A9f..63AE Notes pool (public)
  to 0xf269..4F43 [reader: Carol's everyday account]
  amount 100.000000 BUCK
  fromCipherHash 0xef94..ed27
  toCipherHash 0xc1ab..7dec
emits SpentCoupledB1
  nullifier 0x288b62..956480
  recipient 0xf269..4F43 [reader: Carol's everyday account]
  issuer 0x74c4..CBD7 Aspen Mutual Credit Union (public)
  face 100.000000 BUCK
  eDepForIss ciphertext
```



B1.6. The credit union knows who cashed its note

Aspen Mutual reads the event and opens the sealed identity with its key: Carol. Like a bank that honours a cheque, it now knows who issued the note (itself, for Bob) and who cashed it. That is the whole of what anyone learns, and it is held by a regulated institution that must answer to the law for it. Aspen Mutual can print a receipt naming the depositor.

```
print(f"Aspen Mutual's key opens it to Carol's M: ")
```

⁵Notes.sol, spendCoupledB1, checks publicIssuerOfCommitment[issuanceCommitment]: B1 redemption deliberately names its note, and only B1 does. A1 and A2 spends carry no note handle.

```
f"{eq(elgamal_decrypt(sealed_for_aspen, A['aspen'].sk), point(CAROL))}\n")
slip = chain.note_receipt("b1", "issuer", b1_mint, b1_spend)
print(TextDriver(48).render(render_receipt(slip)))
```

Aspen Mutual's key opens it to Carol's M: True

```

      ALBERTA  BUCK
    P A Y M E N T  R E C E I P T
-----
Receipt  qtwexs6zf2rn
-----
FROM (payer - you) (PUBLIC IDENTITY)
Aspen Mutual Credit Union
Credit Union Charter - Alberta, Canada
acct 0x74c471375761d78bb737cea3c96488631cf9cbd7
idpt 0x0f4f2ffd03cc..5066116aaa27
-----
TO (payee) (PRIVATE IDENTITY)
Carol Nakamura
Alberta Identity - Alberta, Canada
acct 0xf269dcd36ce4672f09f804e7b808f32029644f43
idpt 0x1eb063312673..8a97e1de516b
-----
TRANSACTION
Type      Note spend - B1 bearer, public issuer
Amount    100.000000  BUCK
When      2026-09-21 14:19  UTC
Chain     1
Block     58
Log       2
Tx        0x21d4f0d10cab..2620f911a17d
Mint      0x07d028b289a1..34b593f611f1
-----
VERIFICATION
status @ issue: VALID
receipt qtwexs6zf2rn
-----
a record, not a bearer instrument
github.com/pjkundert/alberta-buck

```

BOB

KNOWN TO HIS BANK

- Bought the note
- Aspen's member records
- ✗ Not on the chain

MALLORY

OUTSIDE OBSERVER

KEEPS ONLY THE PUBLIC LINK

- Aspen note → account
- 🔒 Carol's sealed M
- ✗ No names

CAROL

NAMED TO THE ISSUER

- Cashed into everyday
- ↳ Her M, to Aspen only
- + Aspen's receipt

Aspen Mutual opens the sealed identity: its note was cashed by Carol

ASPEN MUTUAL

Knows who bought its note and who cashed it, as a bank does.
A regulated institution: its records answer to the law.

A1: a draft payable to Carol

An A1 note is a bank draft made out to one person: still issued by a public institution, but sealed to Carol's mailbox. A copied note is useless to anyone else. Carol can deposit it into any of her accounts without telling anyone which in advance, and the cashing names no note.

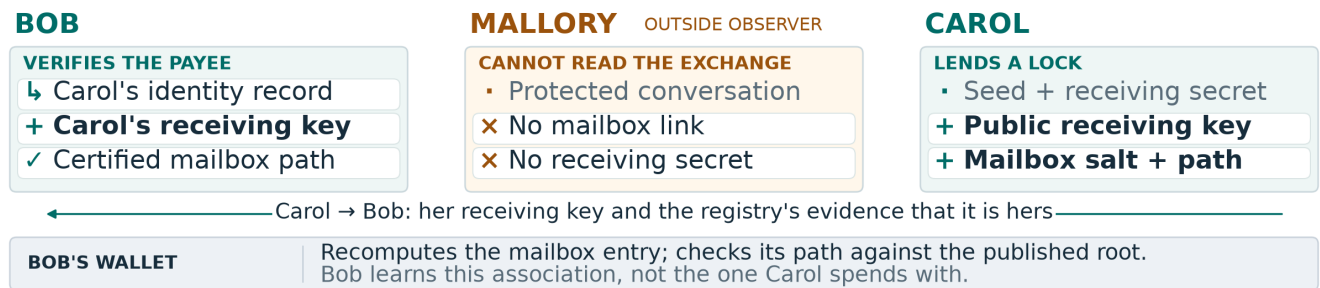
A1.1. Carol gives Bob her mailbox

Carol gives Bob her public receiving key and the registry's evidence that it belongs to her certified identity. Bob, or his credit union, checks that evidence against the published identity root. A public key is a lock

Carol lends out; the key that opens it never leaves her wallet.

```
from alberta_buck.wallet.recvkey import verify_receiving_binding
mailbox = world.mailbox_binding() # Carol's key, a salt, and a path to the root
print(f"this mailbox is certified as Carol's: "
      f"{verify_receiving_binding(point(CAROL), mailbox, world.identity_root)}")

this mailbox is certified as Carol's: True
```



A1.2. The credit union seals the draft

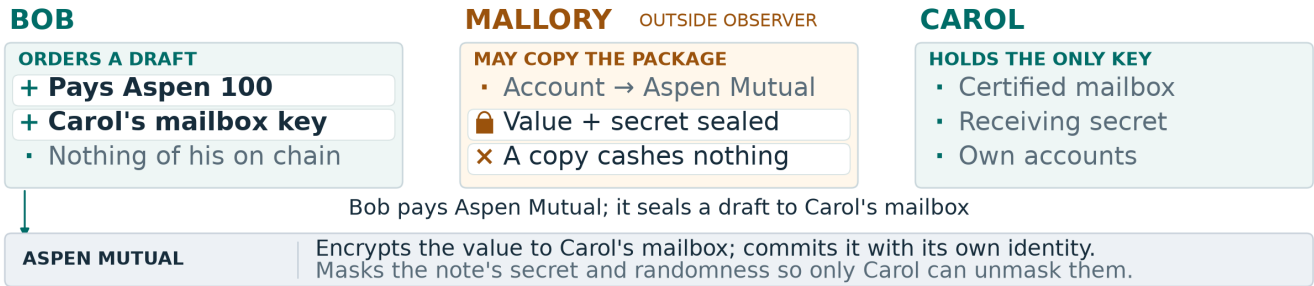
Aspen Mutual encrypts the value to Carol's mailbox and builds the note around it. Everything Carol will need later, the note's secret and the randomness, travels in a *delivery package*, each field masked so that only her receiving secret can unmask it. A courier, or Mallory, can carry the package and learn nothing from it.

```
from alberta_buck.wallet.unilateral_a1 import mint_unilateral_a1
from alberta_buck.wallet.delivery import deliver_a1, open_a1
a1 = world.notes["a1"]
draft = mint_unilateral_a1(carol.M, carol.pk_recv, v=100 * U, rho=a1.opening.rho,
                           m_issuer=P["aspen"].m, rng=world.replay("a1", "mint"))
package = deliver_a1(draft, carol.pk_recv)
print(f"the draft the proofs were made over: {draft.cm == a1.cm}")
card(cards.delivery(package, "a1", "Carol", "Aspen Mutual"), "a1-delivery")
```

```
the draft the proofs were made over: True
```

ADDRESSED DELIVERY (A1)				Aspen Mutual → Carol
SEALED → Carol	value ciphertext	R 0x1a1..35d C 0x121..a8e	Enc(value, recipient's mailbox)	
SEALED → Carol	recipient ciphertext	R 0x217..21f C 0x105..41c	Enc(recipient's M, mailbox)	
SEALED → Carol	value	0x1fc0..8fe9	masked with the mailbox secret	
SEALED → Carol	bearer secret	0x263d..7d28	masked with the mailbox secret	
SEALED → Carol	value randomness	0x1c93..bff7	masked with the mailbox secret	

Only the recipient's receiving secret opens the sealed fields.

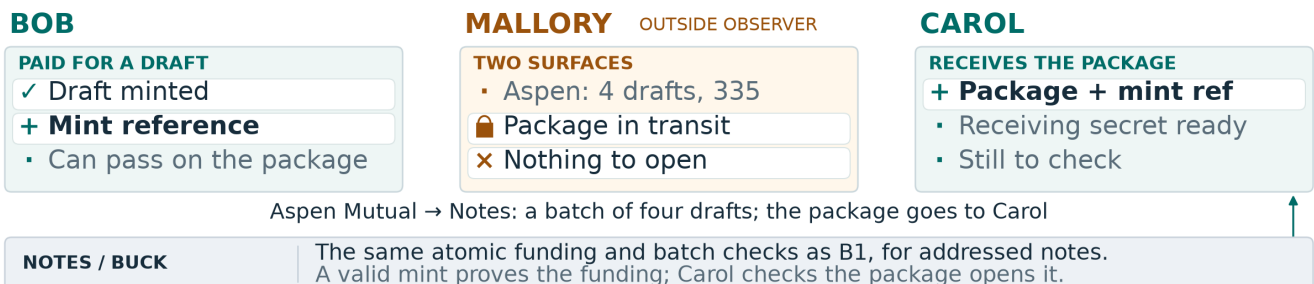


A1.3. Bob pays; the credit union mints

Bob pays Aspen Mutual for the draft, and it mints a batch of four drafts for four payees. Mallory sees 335 BUCK go into the pool from Aspen Mutual. The package goes to Carol; if Mallory intercepts it, she holds sealed numbers.

```
chain.transfer(A["bob"], A["aspen"].address, 100 * U)
a1_mint = chain.mint_note(a1)
print(chain.observe(a1_mint).render())
```

```
Notes.mint from 0x74c4..CBD7 Aspen Mutual Credit Union (public)
tx 0x13bc..b516 block 60 gas 593,703
proof 256 bytes (Groth16 proof)
oldRoot 0x1c024f..66f7bb
newRoot 0x38972d..6a0653
nextLeafIndex_ 4
totalFace 335.000000 BUCK
cms [0x1390ff..71ab25, 0x291607..cd7717, 0x258535..cf2bc2, 0x1662cc..d128fe]
issuerMode [1, 1, 1, 1]
issuerSig issuer signature
emits Transfer
  from 0x74c4..CBD7 Aspen Mutual Credit Union (public)
  to 0x9A9f..63AE Notes pool (public)
  value 335.000000 BUCK
emits BuckTransferReceipt
  from 0x74c4..CBD7 Aspen Mutual Credit Union (public)
  to 0x9A9f..63AE Notes pool (public)
  amount 335.000000 BUCK
  fromCipherHash 0x9e6f..4aaf
  toCipherHash 0x9de4..f456
emits Minted
  issuer 0x74c4..CBD7 Aspen Mutual Credit Union (public)
  newRoot 0x38972d..6a0653
  totalFace 335.000000 BUCK
  startIndex 4
  count 4
emits IssuerBound
  issuer 0x74c4..CBD7 Aspen Mutual Credit Union (public)
  newRoot 0x38972d..6a0653
```



A1.4. Carol unpacks it

Carol's receiving secret unmask the fields. Her wallet checks the value really is 100, that the unpacked note rebuilds a commitment in Aspen Mutual's batch, and that it is unspent. Any other key opens nothing.

```
from alberta_buck.wallet.delivery import DeliveryRefused
got = open_a1(package, carol.k, P["aspen"].m)
print(f"value {got.opening.v / U:.2f} BUCK; commitment in Aspen Mutual's batch: {got.cm in a1.cms}")
try:
    open_a1(package, rand_scalar(rng), P["aspen"].m)
except DeliveryRefused as refused:
    print(f"with any other key: refused ({refused})")
```

```
value 100.00 BUCK; commitment in Aspen Mutual's batch: True
with any other key: refused (the unwrapped r_note is not eNote's randomness)
```

BOB

KEEPS HIS RECORD

- Paid Aspen 100
- Knows the payee
- ✗ Cannot cash it

MALLORY OUTSIDE OBSERVER

HAS ONLY A SEALED COPY

- 🔒 Masked value + secret
- ✗ No key from any ID
- ✗ Tampering fails

CAROL

UNPACKS THE DRAFT

- ↳ Package → 100 + secret
- ✓ Value really 100
- ✓ Note in Aspen's batch

Carol's secret → unmasked fields → rebuilt note → verified mint

CAROL'S WALLET

Checks value, randomness, issuer, funding and spent status.
Any other key opens nothing.

A1.5. One proof that the pieces are all hers

Carol proves two things at once: she can read this note, and she is the certified person it is addressed to. Proved separately, a thief could satisfy the first with a stolen package and the second with his own identity. So they are *folded* into one proof over one set of secrets: her receiving secret opens the note, her account holds the certified identity, and the registry's sealed entry ties that identity to that receiving secret.

The proof takes a laptop about 15 seconds. It shows the chain what the chain needs, and nothing else.⁶

```
fresh = cards.short_ct(a1.e_enc) # her identity, re-sealed for this spend
card(cards.Card("A1 DEPOSIT: THE FOLDED PROOF", "proved by Carol's wallet",
    note=f"Proved in advance in {a1.timings['deposit_gate_prove_s']:.1f}s; "
    "the chain verifies it in one step.").add(
    F("spent marker", cards.short_int(a1.nullifier), "public", "", "refuses a second cashing"),
    F("value, payout", f"100.00 BUCK -> {cards.short_addr(A['carol'].address)}", "public"),
    F("identity root", cards.short_int(world.identity_root), "public", "", "current certification"),
    F("fresh envelope", fresh, "sealed", "Carol", "her M, newly re-sealed"),
    F("which note", "not shown", "proven", "", "one of the pool's notes; no handle"),
    F("receiving secret", "used, not shown", "proven", "", "opens this note"),
    F("certified identity", "used, not shown", "proven", "", "her account's; tied to her key"),
    F("account key", "used, not shown", "secret", "Carol", "never leaves her wallet")), "a1-fold")
```

⁶Notes Flow: current spend paths, delivery.py, A1 folded gate and A2 folded gate.

A1 DEPOSIT: THE FOLDED PROOF			proved by Carol's wallet
PUBLIC	spent marker	0xb96a..cd6c	refuses a second cashing
PUBLIC	value, payout	100.00 BUCK -> 0xf269..4F43	
PUBLIC	identity root	0x2243..84cb	current certification
SEALED → Carol	fresh envelope	R 0x1fe..461 C 0x198..1d9	her M, newly re-sealed
PROVEN	which note	not shown	one of the pool's notes; no handle
PROVEN	receiving secret	used, not shown	opens this note
PROVEN	certified identity	used, not shown	her account's; tied to her key
SECRET: Carol	account key	used, not shown	never leaves her wallet

Proved in advance in 14.6s; the chain verifies it in one step.

BOB

NOT NEEDED

- Paid; knows the payee
- Holds no Carol keys
- ✗ Cannot prove as Carol

MALLORY

OUTSIDE OBSERVER

GETS VERIFIABLE CLAIMS

- Marker + face
- 🔒 Fresh identity envelope
- ✗ No note, leaf or key

CAROL

PROVES IT IS ALL HERS

- Note + receiving secret
- Account + live entry
- + Spend + folded proofs

Carol → Notes: a note proof plus one folded identity-and-mailbox proof

FOLDED DEPOSIT GATE

One witness ties note key, identity, certified entry and account.
Proved separately, a thief could supply one of each.

A1.6. Cashed, with no handle on the note

The contract checks the proofs and pays Carol. Compare what Mallory sees with B1: no note, no issuer, only that someone cashed 100 BUCK from the pool into this account. Aspen Mutual issued four drafts, and the chain does not say which this was. Bob and Carol can still recognise their own note; privacy from strangers is not amnesia for the parties.

```
a1_spend = chain.spend_note(a1)
print(chain.observe(a1_spend).render())
```

```
Notes.spendCoupledA1 from 0xf269..4F43 [reader: Carol's everyday account]
tx 0x86d7..8321 block 61 gas 874,877
proof 256 bytes (Groth16 proof)
root 0x38972d..6a0653
identityRoot 0x2243cb..b184cb
nullifier 0xb96a4f..c9cd6c
face 100.000000 BUCK
recipient 0xf269..4F43 [reader: Carol's everyday account]
eRec ciphertext
foldProof 256 bytes (Groth16 proof)
emits Transfer
  from 0x9A9f..63AE Notes pool (public)
  to 0xf269..4F43 [reader: Carol's everyday account]
  value 100.000000 BUCK
emits BuckTransferReceipt
  from 0x9A9f..63AE Notes pool (public)
  to 0xf269..4F43 [reader: Carol's everyday account]
  amount 100.000000 BUCK
  fromCipherHash 0xef94..ed27
  toCipherHash 0xc1ab..7dec
```

```

emits SpentCoupledA1
nullifier      0xb96a4f..c9cd6c
recipient      0xf269..4F43 [reader: Carol's everyday account]
face           100.000000 BUCK

```

BOB

CAN RECOGNISE IT

- The draft he paid for
- ✓ Cashed, by Carol
- + **Receipt, if he wants**

MALLORY OUTSIDE OBSERVER

SEES A POOL PAYOUT

- 100 → an account
- Time + proofs + marker
- ✗ No note, no issuer

CAROL

PAID; KEEPS EVIDENCE

- ✓ All checks passed
- ✓ 100 BUCK credited
- + **Receipt + chain anchor**

Notes validates the draft, consumes the marker and pays Carol

NOTES / BUCK

Checks roots, standing, face, payout, note proof and folded proof.
No note handle: which of Aspen's four drafts is not on the chain.

A2: Bob's own private cheque

Now Bob writes the note himself, from his own account, with no institution in between. His identity travels inside the note, sealed to Carol's mailbox. Only Carol will learn who wrote it.

A2.1. Bob seals his identity inside the note

Bob encrypts the value and his own identity point to Carol's mailbox. He adds a proof that the sealed identity is the one his account registered, and a *key tie*, marked **T**, that will force Carol's later proof to use the mailbox Bob actually addressed. Without it, a dishonest issuer could make a note that names someone other than himself.

```

from alberta_buck.wallet.unilateral_a2 import mint_unilateral_a2
from alberta_buck.wallet.delivery import deliver_a2, open_a2
a2 = world.notes["a2"]
cheque = mint_unilateral_a2(A["bob"].sk, A["bob"].E, carol.pk_recv, v=100 * U,
                             rho=a2.opening.rho, issuer=A["bob"].addr, chainid=1,
                             salt_iss=bob.salts["naming"], rng=world.replay("a2", "mint"))
print(f"the cheque the proofs were made over: {cheque.cm == a2.cm}")
print(f"Bob's identity, sealed for Carol:      {cards.short_ct(cheque.eIss)}")
print(f"key tie T:                            {cards.short_point(cheque.binding.T)}")

```

```

the cheque the proofs were made over: True
Bob's identity, sealed for Carol:      R 0x661..bd2  C 0x137..ace
key tie T:                            (0x289..871, 0xc69..3db)

```

BOB

BUILDS A PRIVATE-ISSUER NOTE

- Carol's certified mailbox
- 🔒 Value + Bob's identity
- + **Issuer binding + tie T**

MALLORY OUTSIDE OBSERVER

SEES A PRIVATE ACCOUNT

- Minting address B
- 🔒 Bob's identity envelope
- ✗ No clear issuer identity

CAROL

PROVIDES THE SAME MAILBOX

- Receiving public key
- Certified association
- Receiving secret private

A2 replaces public Bob with an envelope only Carol's mailbox can open

BOB'S WALLET

Proves encrypted issuer identity equals the one in his registered account.
Adds a hidden key tie T; the note commits to it as well as the ciphertexts.

A2.2. The package carries its own tools, sealed

Carol will need more than the ciphertexts: the note's secret, the randomness Bob used, the blind that opens **T**, and the salt of Bob's registry entry that lets her prove he is certified. Each is masked for her alone. Sending any of them in the clear would reveal Bob by another route.

```
package2 = deliver_a2(chèque, carol.pk_recv)
card(cards.delivery(package2, "a2", "Carol", "Bob"), "a2-delivery")
```

ADDRESSED DELIVERY (A2)				Bob → Carol
SEALED → Carol	value ciphertext	R 0x21e...e01 C 0x1b6...184	Enc(value, recipient's mailbox)	
SEALED → Carol	issuer ciphertext	R 0x661...bd2 C 0x137...ace	Enc(Bob's M, mailbox)	
PUBLIC	key tie T	(0x289...871, 0xc69...3db)		blinded; opens only in proof
SEALED → Carol	value	0x1d50...1df2		masked with the mailbox secret
SEALED → Carol	bearer secret	0x171a...7983		masked with the mailbox secret
SEALED → Carol	issuer randomness	0x1aa5...f53c		masked with the mailbox secret
SEALED → Carol	key-tie blind	0x1c56...daaf		masked with the mailbox secret
SEALED → Carol	issuer's naming salt	0x170a...015a		masked with the mailbox secret
Only the recipient's receiving secret opens the sealed fields.				

BOB

SEALS THE INNER MATERIAL

- Value + identity envelopes
 - Binding + key tie T
- 🔒 Opening + naming material

MALLORY OUTSIDE OBSERVER

CAN COPY, NOT UNWRAP

- Ciphertexts + T
- 🔒 Each secret field masked
- ✗ No opening or naming salt

CAROL

CAN REBUILD THE WRAP KEY

- Receiving secret
 - Public envelope component
- + Shared wrap material

Bob derives wrapping material from Carol's public key; Carol derives it locally

ADDRESSED DELIVERY

Wraps value, opening secret, issuer randomness, naming salt and tie-opening blind. Each field has its own mask. None of these disclosure keys travels in the clear.

A2.3. Bob mints

Bob's account introduces itself to the pool, allows it 100 BUCK, and mints. The contract checks the note proof and Bob's binding and moves his money into the pool. Mallory sees an account mint one 100 BUCK note with a sealed issuer. If she already knows the account is Bob's, she knows Bob wrote a note; she does not learn to whom.

```
chain.approve(chain.envelope(A["bob"], chain.notes.address, chain.pool_pk, rng), 100 * U)
a2_mint = chain.mint_note(a2)
print(chain.observe(a2_mint).render())
```

```
Notes.mint from 0x01f9..564d [reader: Bob's everyday account]
tx 0x8836..6955 block 63 gas 567,449
proof                256 bytes (Groth16 proof)
oldRoot              0x38972d..6a0653
newRoot              0x26db54..8e68b0
nextLeafIndex_      8
totalFace            100.000000 BUCK
```

```

cms                [0x1e9640...c0241c]
issuerMode         [2]
a2Bindings         1 x issuer binding
emits Transfer
  from              0x01f9..564d   [reader: Bob's everyday account]
  to                0x9A9f..63AE Notes pool (public)
  value             100.000000 BUCK
emits BuckTransferReceipt
  from              0x01f9..564d   [reader: Bob's everyday account]
  to                0x9A9f..63AE Notes pool (public)
  amount            100.000000 BUCK
  fromCipherHash    0xa3b8..7eef
  toCipherHash      0xf47c..115d
emits Minted
  issuer            0x01f9..564d   [reader: Bob's everyday account]
  newRoot           0x26db54..8e68b0
  totalFace         100.000000 BUCK
  startIndex        8
  count             1
emits IssuerReencBound
  issuer            0x01f9..564d   [reader: Bob's everyday account]
  newRoot           0x26db54..8e68b0
  count             1

```

BOB

MINTS HIS CHEQUE

- Sealed A2 delivery

+ **Mint + issuer proofs**

✓ 100 BUCK escrowed

MALLORY

OUTSIDE OBSERVER

READS PUBLIC FUNDING

- Account → pool: 100
- Commitment + envelope
- Blinded proofs + T

CAROL

CAN CHECK THE PUBLIC ANCHOR

- Receiving secret

+ **Mint reference**

- Still needs delivery

Bob → Notes: commitments, encrypted issuer data, key ties and checked proofs

NOTES / BUCK

Checks A2 mode, value conservation, issuer binding and committed ciphertext ties. Escrows the batch atomically. Minting address and total remain public.

A2.4. Carol opens it and learns who wrote it

Carol unmarks the package with her receiving secret, decrypts the issuer's identity, and checks it against Bob's record. She checks the key tie against her own mailbox, and finds Bob's certified registry entry using the salt he shipped. That salt lets her prove Bob is certified; it gives her nothing that opens his mail.

```

from alberta_buck.wallet.nums import H_PEDERSEN
got2 = open_a2(package2, carol.k)
tie = add(add(got2.eIss.C, neg(got2.T)), mul(H_PEDERSEN, got2.gamma))
print(f"value {got2.opening.v / U:.2f} BUCK, written by Bob: {eq(got2.M_I, point(BOB))}")
print(f"the key tie opens to the same identity: {eq(tie, got2.M_I)}")
print(f"Bob's naming entry is in the registry: "
      f"{tree.contains(identity_leaf_salted(got2.M_I, got2.salt_iss))}")

```

```

value 100.00 BUCK, written by Bob: True
the key tie opens to the same identity: True
Bob's naming entry is in the registry: True

```

BOB

DELIVERS THE COMPLETE NOTE

- Funded A2 commitment

🔒 Blob + identity record

+ **Mint evidence attached**

MALLORY

OUTSIDE OBSERVER

CAN OBSERVE DELIVERY

- Traffic, size and timing
- 🔒 Inner naming material
- ✗ No Bob identity from blob

CAROL

OPENS IN LAYERS

↳ Value + note opening

↳ Issuer envelope → M_B

✓ Record + T + mint match

Bob → Carol: funded package; Carol unwraps, decrypts and cross-checks each layer

CAROL'S WALLET

Checks 100, issuer randomness, T against her mailbox, commitment and funding. Checks Bob's canonical record against M_B and his separate naming membership.

A2.5. The folded proof grows two checks

Carol’s proof is A1’s, plus two things: the identity she decrypted belongs to a certified person, and the key tie opens with her own mailbox. The first stops a cheque signed by an invented person; the second, a cheque that names a real person who did not write it. "Someone certified" and "the person who wrote this" are different claims, and the proof makes both.

```
card(cards.Card("A2 DEPOSIT: THE FOLDED PROOF", "proved by Carol's wallet",
    note=f"Proved in advance in {a2.timings['deposit_gate_prove_s']:.1f}s.").add(
    F("spent marker", cards.short_int(a2.nullifier), "public", "", "refuses a second cashing"),
    F("value, payout", f"100.00 BUCK -> {cards.short_addr(A['carolSavings'].address)}", "public",
        "", "into her savings account"),
    F("fresh issuer envelope", "re-sealed", "sealed", "Carol", "Bob's M, newly re-sealed"),
    F("which note", "not shown", "proven", "", "one of the pool's notes; no handle"),
    F("her identity and key", "used, not shown", "proven", "", "as in A1"),
    F("issuer certified", "used, not shown", "proven", "", "Bob's naming entry"),
    F("key tie T", "used, not shown", "proven", "", "opens with her mailbox")), "a2-fold")
```

A2 DEPOSIT: THE FOLDED PROOF			proved by Carol's wallet
PUBLIC	spent marker	0x2deb..84b0	refuses a second cashing
PUBLIC	value, payout	100.00 BUCK -> 0x11c3..947F	into her savings account
SEALED → Carol	fresh issuer envelope	re-sealed	Bob's M, newly re-sealed
PROVEN	which note	not shown	one of the pool's notes; no handle
PROVEN	her identity and key	used, not shown	as in A1
PROVEN	issuer certified	used, not shown	Bob's naming entry
PROVEN	key tie T	used, not shown	opens with her mailbox

Proved in advance in 14.6s.

BOB
HAS PROVIDED NAMING EVIDENCE

- Encrypted own identity
- Naming salt, wrapped
- Opening of T, wrapped

MALLORY OUTSIDE OBSERVER
SEES PROOFS, NOT WITNESSES

- Accepted identity root
- Spent marker + proofs
- ✗ No secret leaf or path

CAROL
BUILDS THE STRONGER FOLD

- My account ↔ my mailbox
- + Bob is a live member
- + T pins the actual issuer

Carol joins the A1 account-and-mailbox checks with two additional A2 checks

A2 FOLDED DEPOSIT GATE	Checks membership of the decrypted issuer and opens T with Carol's own key. Being a certified identity is one claim; being this note's actual issuer is another.
-------------------------------	--

A2.6. Cashed into her savings

Carol cashes the cheque into her savings account, a different account from the one Bob paid before. The spend carries a fresh re-sealing of Bob’s identity, never the one from the mint, so nothing in it matches the mint. Mallory sees 100 BUCK leave the pool for an account she has never seen pay or be paid by Bob.

```
a2_spend = chain.spend_note(a2)
print(chain.observe(a2_spend).render())
print(f"\nCarol's accounts: everyday {chain.balance(A['carol'].address) / U:,.2f} BUCK, "
    f"savings {chain.balance(A['carolSavings'].address) / U:,.2f} BUCK")
```

```

Notes.spendCoupledA2 from 0x11c3..947F [reader: Carol's savings account]
tx 0xba82..fda4 block 64 gas 886,060
proof          256 bytes (Groth16 proof)
root           0x26db54..8e68b0
identityRoot   0x2243cb..b184cb
nullifier      0x2debcd..9984b0
face          100.000000 BUCK
recipient      0x11c3..947F [reader: Carol's savings account]
eIss           ciphertext
foldProof      256 bytes (Groth16 proof)
emits Transfer
  from         0x9A9f..63AE Notes pool (public)
  to           0x11c3..947F [reader: Carol's savings account]
  value        100.000000 BUCK
emits BuckTransferReceipt
  from         0x9A9f..63AE Notes pool (public)
  to           0x11c3..947F [reader: Carol's savings account]
  amount       100.000000 BUCK
  fromCipherHash 0xa914..a7af
  toCipherHash  0x9b1e..6a82
emits SpentCoupledA2
  nullifier     0x2debcd..9984b0
  recipient     0x11c3..947F [reader: Carol's savings account]
  face         100.000000 BUCK

```

Carol's accounts: everyday 300.00 BUCK, savings 100.00 BUCK

BOB

RETAINS THE ORIGINAL NOTE

- Mint envelope + T
- Known opening → marker
- Carol's identity record

MALLORY

OUTSIDE OBSERVER

READS A FRESH PAYOUT RECORD

- 100 → payout account
- 🔒 Fresh issuer envelope
- ✗ No repeated mint handle

CAROL

SUBMITS; THEN RECEIVES

- + Fresh envelope + proofs
- ✓ 100 BUCK credited
- ✓ Spent marker recorded

Carol submits a fresh re-encryption; Notes validates and pays her account

NOTES / BUCK

Checks note, flavour, roots, unused marker and the whole folded statement.
Pays atomically. The original envelope and T stay hidden inside the spend proof.

A2.7. Two receipts, one payment

Each keeps a receipt naming both. Carol's rests on what only she can prove, that her receiving secret opens the note; Bob's on what only he knows, the randomness he sealed it with. Neither receipt contains a key that opens anything else.⁷

```

def side_by_side(a, b, gap=4):
    la, lb = a.rstrip().split("\n"), b.rstrip().split("\n")
    w, n = max(map(len, la)), max(len(la), len(lb))
    la += [""] * (n - len(la)); lb += [""] * (n - len(lb))
    return "\n".join(f"{x:<{w}}{' ' * gap}{y}" for x, y in zip(la, lb))

carol_copy = chain.note_receipt("a2", "recipient", a2_mint, a2_spend)
bob_copy = chain.note_receipt("a2", "issuer", a2_mint, a2_spend)
render = lambda core: TextDriver(48).render(render_receipt(core))
print(side_by_side(render(carol_copy), render(bob_copy)))

```

ALBERTA BUCK P A Y M E N T R E C E I P T	ALBERTA BUCK P A Y M E N T R E C E I P T
-----	-----
Receipt okrxtssonrka	Receipt i6nr27ezzu7v
-----	-----

⁷Receipts specifies the receipt format and its two verification tiers. A receipt names both parties and should be kept as deliberately as any other private financial record.

```

FROM (payer) (PRIVATE IDENTITY)
  Bob Tremblay
  Alberta Identity - Alberta, Canada
  acct 0x01f9fec6bb2f227da0fe180651c1616e0f03564d
  idpt 0x110f449d8e6d..6a6925e3ced6
-----
TO (payee - you) (PRIVATE IDENTITY)
  Carol Nakamura
  Alberta Identity - Alberta, Canada
  acct 0x11c38344ef77494c5e5aaa11ec72fe9dd5e0947f
  idpt 0x1eb063312673..8a97e1de516b
-----
TRANSACTION
Type   Note spend - A2 addressed, private issuer
Amount           100.000000 BUCK
When           2026-09-21 14:20 UTC
Chain                1
Block               64
Log                 2
Tx      0xba82b328f4bc..40aba205fda4
Mint     0x8836c2c14b87..a1e2ff976955
-----
VERIFICATION
status @ issue: VALID
receipt okrxxtssonrka
-----
a record, not a bearer instrument
github.com/pjkundert/alberta-buck

```

```

FROM (payer - you) (PRIVATE IDENTITY)
  Bob Tremblay
  Alberta Identity - Alberta, Canada
  acct 0x01f9fec6bb2f227da0fe180651c1616e0f03564d
  idpt 0x110f449d8e6d..6a6925e3ced6
-----
TO (payee) (PRIVATE IDENTITY)
  Carol Nakamura
  Alberta Identity - Alberta, Canada
  acct 0x11c38344ef77494c5e5aaa11ec72fe9dd5e0947f
  idpt 0x1eb063312673..8a97e1de516b
-----
TRANSACTION
Type   Note spend - A2 addressed, private issuer
Amount           100.000000 BUCK
When           2026-09-21 14:20 UTC
Chain                1
Block               64
Log                 2
Tx      0xba82b328f4bc..40aba205fda4
Mint     0x8836c2c14b87..a1e2ff976955
-----
VERIFICATION
status @ issue: VALID
receipt i6nr27ezzu7v
-----
a record, not a bearer instrument
github.com/pjkundert/alberta-buck

```

BOB

KEEPS PAYER-SIDE EVIDENCE

- Identity + mailbox binding
- Opening + randomness
- ✓ Confirmed spend anchor

MALLORY OUTSIDE OBSERVER

STILL HAS THE PUBLIC SURFACE

- Funding + payout records
- Timing and value clues
- ✗ No private receipt contents

CAROL

KEEPS RECIPIENT EVIDENCE

- Both identity records
- + Verifiable decryption
- ✓ Mint + spend + key tie

Each participant builds evidence from what they hold; Carol keeps her secret key

PRIVATE RECEIPTS

Names + note opening + relevant proofs + authenticated chain references.
Payer uses mint evidence; recipient proves decryption. Neither exports wallet keys.

Evidence for those entitled to it

R1. Carol discloses one receipt

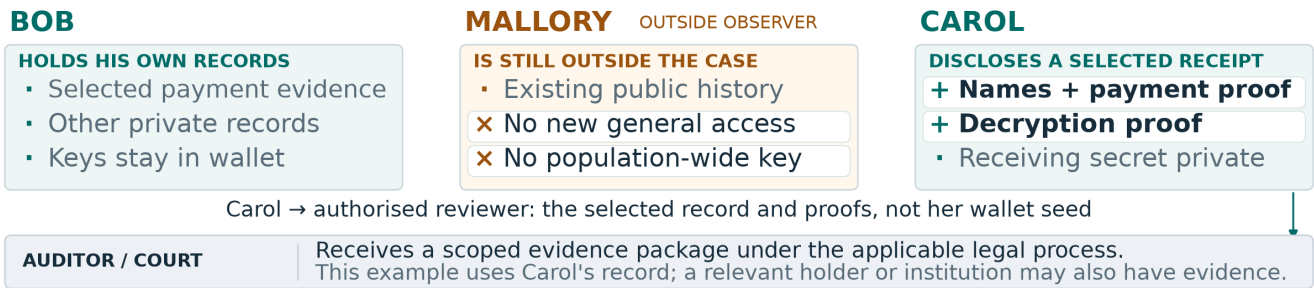
Suppose the payment is disputed. Carol hands a reviewer, a court or an auditor, the one receipt that matters. She does not hand over her seed, her keys or her other accounts, and the receipt contains none of them. The reviewer checks it offline first: the names match the identity points, and every proof inside it holds.

```

from alberta_buck.wallet.envelope import deserialize_core, receipt_id, serialize_core
from alberta_buck.wallet.verify_receipt import verify_receipt
evidence = serialize_core(carol_copy) # the bytes printed on the slip
checked = verify_receipt(deserialize_core(evidence))
print(f"receipt {receipt_id(evidence)}: {checked.reason}, "
      f"{checked.value / U:.2f} BUCK, {len(evidence):,} bytes")

```

```
receipt okrxxtssonrka: VALID, 100.00 BUCK, 11,587 bytes
```

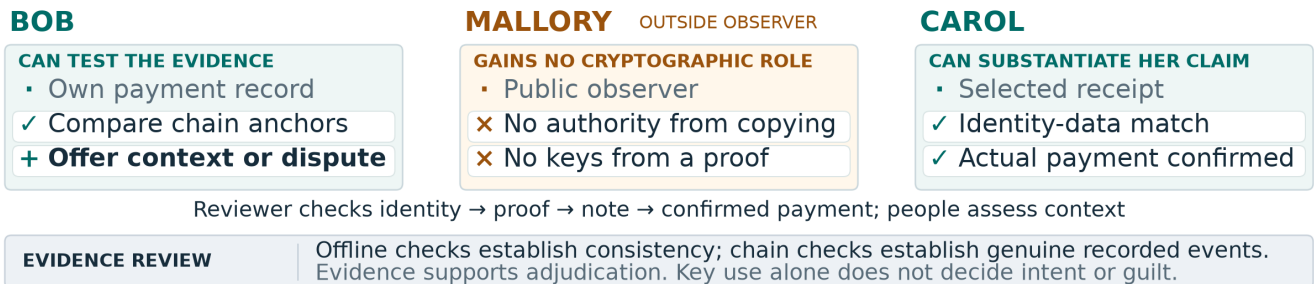


R2. The chain confirms it happened

Then the reviewer asks the chain: did this transaction occur, at this block, with this event, from these registered accounts? A convincing slip is not a payment; the chain's record is. What the evidence cannot settle, intent, a stolen key, a false certification, is for people to judge. Good cryptography gives them better facts, and leaves citizens room for ordinary private life.

```
tx = anvil.w3.eth.get_transaction_receipt(carol_copy.txn.txhash)
print(f"spend mined in block {tx['blockNumber']} (receipt says {carol_copy.txn.block}); "
      f"event #{carol_copy.txn.logindex} present: "
      f"{carol_copy.txn.logindex in [log['logIndex'] for log in tx['logs']]}")
registered = chain.reg.functions.pkOf(A["carolSavings"].address).call()
print(f"payee account registered with the receipt's key: "
      f"{registered[0] == int(carol_copy.payee.pk['x'], 16)}")
```

```
spend mined in block 64 (receipt says 64); event #2 present: True
payee account registered with the receipt's key: True
```



What Mallory can join

The gain is choice over who learns what, not an invisible ledger. This table is Mallory's view from the chain alone, as the transactions above printed it.

Payment	What the chain connects	Names on the chain	Who can recognise the payment
Direct	Bob's account to Carol's; amount; time	none (both envelopes sealed)	Bob, Carol
B1	an Aspen Mutual note to Carol's account	Aspen Mutual (public)	Bob, Carol, Aspen Mutual
A1	Aspen Mutual into the pool; the pool to Carol's account	Aspen Mutual (public)	Bob, Carol, Aspen Mutual
A2	Bob's account into the pool; the pool to Carol's savings	none	Bob, Carol

A few boundaries keep the claim honest:

- **Counterparties know their own dealings.** Bob can recognise his note when it is cashed; Carol knows who paid her. That is what makes receipts possible.
- **Institutions see their customers.** Aspen Mutual knows Bob bought a note and Carol cashed one; the pool's operator knows who uses the pool. They answer to regulators for that knowledge.
- **Standing is current.** Removing someone from the registry stops new proofs of their standing; Notes accept an identity root up to seven days old.
- **Behaviour leaks.** An unusual amount, a quiet pool, cashing the moment a note is minted, or paying gas from a linked account can suggest a connection the cryptography does not make.
- **Habits help.** Standard amounts, like banknote denominations, make one note look like many. An addressed note is safe to hold, so cashing it days later separates the cashing from the minting. A bearer note is safer cashed promptly, so its cover is a busy issuer and a common denomination. These are the habits of any privacy pool, mixers included. The difference is who can use this one: every depositor is a certified person, named to the counterparty and, for a bearer note, to its issuer.
- **Keys matter.** Lose the seed and you lose the receiving key; leak it and old deliveries sealed to it can be read. A weakness in the underlying cryptography, or a compromised wallet, is outside these promises.

Let strangers verify that the rules were followed, let citizens know and prove their own dealings, and let public institutions examine evidence through accountable process. Buying a bicycle should not require publishing the rest of your life.

```
anvil.stop()
print("chain stopped; every receipt above still verifies offline.")
```

```
chain stopped; every receipt above still verifies offline.
```

Sources and reading notes

This account follows the current v2 design and its prototype, checked against the code it runs. It is not a description of an operating Alberta government service.

Illustration sources

The step diagrams are generated from the figure descriptions by `render_privacy_figures.py`; the payload cards by `cards.py` as the paper runs. Each picture has an alternative description, and the surrounding step explains its contents.